Move семантики

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Move семантики - ползи, lvalue, rvalue, move конструктор/оп=, std::move. Пример за клас стринг с move семантика. (C++ 11). Масиви от указатели към обекти.

Какво са move semantics?

Move позволява прехвърляне на ресурс (pointer) от един обект в друг – почти O(1).

Копирането на големи буфери е скъпо (O(n)).

Подобрява производителността при контейнери (например std::vector) и фабрични функции.

Намалява алокации и освобождавания на памет.

Пример

```
SimpleString a("Very very long string ...");

SimpleString b = a; // copy ctor → заделя нов буфер и копира всички символи
SimpleString c = std::move(a); // move ctor → прехвърля само указателя
```

При копиране: нов new[] + memcpy.

При move: само присвояване на указател и зануляване на стария.

Какво са lvalues и rvalues?

lvalue: стойност с име/адрес – може да има длъжност да бъде променяна. Пример: x, obj.member.

rvalue: временна стойност, няма дългосрочен адрес. Пример: 42, x + y, std::string("tmp").

съществуват още xvalues, prvalues, glvalues

Пример

```
int x = 10; // x е lvalue
int y = x; // x се използва като lvalue

int z = x + y; // (x + y) е rvalue


SimpleString s1("Hello"); // s1 е lvalue
SimpleString s2 = s1; // копиране (lvalue → copy ctor)


SimpleString s3 = SimpleString("Temp"); // временен
обект → rvalue
```

rvalue reference (T&&)

Какво е rvalue reference?

T&& може да се свърже само с rvalue (временни обекти).

Основният механизъм зад move семантиката.

Позволява "източване" на ресурси от временни обекти.

Примери

```
void takeByRvalue(SimpleString&&
s) {
// s е rvalue reference → можем
да му "вземем" ресурсите
SimpleString local = std::move(s);
}

int main() {
SimpleString a("Hello");

// takeByRvalue(a); // ❌ грешка: a
е lvalue
takeByRvalue(std::move(a)); // ✅
превръщаме a в rvalue

takeByRvalue(SimpleString("Temp"))
; // ✅ временен обект (rvalue)
}
```

Допълнение

rvalue reference не означава временен живот — s има име и е lvalue вътре във функцията.

Затова често се използва std::move(s) вътре в тялото.

std::move - примери

```
Какво прави std::move -

std::move не мести нищо сам по себе си.
Той прави cast към rvalue reference: static_cast<T&&>(obj).
След std::move обектът е валиден, но със неопределено съдържание


SimpleString a("Hello");
SimpleString b = std::move(a); // извиква move ctor
// a е валиден, но съдържанието му не трябва да се използва

SimpleString a("Hello");
SimpleString b;
b = std::move(a); // извиква move operator=

void foo(SimpleString s);

SimpleString x("Test");
foo(x); // copy ctor (x е lvalue)
foo(std::move(x)); // move ctor (x е cast-нат до rvalue)
```

Move Constructor

```
При инициализация от rvalue.
При връщане на стойност от функция (ако няма/не се приложи RVO).
При работа с контейнери и временни обекти.

SimpleString make() {
SimpleString tmp("Hello");
return tmp; // RVO или move ctor
}

SimpleString a = make();

SimpleString a("A");
SimpleString b(std::move(a)); // директно извикване на move ctor

std::vector<SimpleString> v;
SimpleString s("Data");

v.push_back(s); // copy ctor
v.push_back(std::move(s)); // move ctor

SimpleString create() {
return SimpleString("Temp"); // временен обект → move или elision
}
```

Move Assignment operator

Използва се при присвояване от rvalue.

Освобождава текущите ресурси и ги взема от other.

Оставя other в валидно, но празно състояние.

noexcept е важно да се слага заради STL контейнери!

Пример

```
SimpleString a("Hello");
SimpleString b("World");
b = std::move(a); // move operator=
// b вече държи "Hello"
// a е валиден, но празен
SimpleString a("A");
SimpleString b("B");
b = a; // copy operator=
b = std::move(a); // move operator=
```

Rule of Five

Ако класът управлява ресурс (heap, файл, mutex и т.н.) и дефинирате един от следните – трябва да дефинирате всички:

Деструктор

Copy конструктор

Copy оператор=

Move конструктор

Move оператор=

Пример

```
class Resource {
public:
Resource();
~Resource(); // освобождава ресурса

Resource(const Resource&); // копира ресурса
Resource& operator=(const Resource&);

Resource(Resource&&) noexcept; // прехвърля
ресурса
Resource& operator=(Resource&&) noexcept;
};
```

Пример за голямата петорка

```
class SimpleString {
public:
SimpleString();
SimpleString(const char* s); // конструктор от C-string
SimpleString(const SimpleString& other); // копиращ
SimpleString(SimpleString&& other) noexcept; // move конструктор
SimpleString& operator=(const SimpleString& other); // копиращ =
SimpleString& operator=(SimpleString&& other) noexcept; // move =
~SimpleString();

private:
char* data_ = nullptr; // менажира буфера
size_t size_ = 0;
};
```

Имплементация на операторите за копиране

```
SimpleString::SimpleString(const SimpleString& other) {
    size_ = other.size_;
    if (size_) {
        data_ = new char[size_ + 1];
        std::memcpy(data_, other.data_, size_ + 1);
    }
}



SimpleString& SimpleString::operator=(const SimpleString& other) {
    if (&other == this) return *this;
    delete[] data_;
    size_ = other.size_;
    data_ = (size_) ? new char[size_ + 1] : nullptr;
    if (data_) std::memcpy(data_, other.data_, size_ + 1);
    return *this;
}
```

Имплементация на move операторите

```
SimpleString::SimpleString(SimpleString&& other) noexcept
: data_(other.data_), size_(other.size_) {
    other.data_ = nullptr;
    other.size_ = 0;
}



SimpleString& SimpleString::operator=(SimpleString&& other)
noexcept {
    if (this == &other) return *this;
    delete[] data_; // освободим наши ресурси
    data_ = other.data_;
    size_ = other.size_;
    other.data_ = nullptr;
    other.size_ = 0;
    return *this;
}
```

Масиви от указатели

```
MyClass* arr[3];
arr[0] = new MyClass(1);
arr[1] = new MyClass(2);
arr[2] = new MyClass(3);


Move семантиката не решава ownership при сурови указатели.


Move на указател = копиране на адрес.


Няма информация кой е отговорен да извика delete.


MyClass* p = new MyClass(42);
MyClass* q = p; // копие на указател


// кой трябва да delete-не p/q?
delete p;
delete q;
```

Масиви от указатели

```
MyClass* arr[3];
arr[0] = new MyClass(1);
arr[1] = new MyClass(2);
arr[2] = new MyClass(3);


Move семантиката не решава ownership при сурови указатели.


Move на указател = копиране на адрес.


Няма информация кой е отговорен да извика delete.


MyClass* p = new MyClass(42);
MyClass* q = p; // копие на указател


// кой трябва да delete-не p/q?
delete p;
delete q;
```

Решение на проблема с ресурсите

Smart pointers, които ще бъдат разгледани по-нататък подробно.

```
std::vector<std::unique_ptr<MyClass>> v;
v.push_back(std::make_unique<MyClass>(1));

std::vector<std::unique_ptr<MyClass>> v2 = std::move(v);
// ownership е коректно прехвърлен
```

Обобщение

Масиви от raw pointer + move ≠ безопасност

Move работи добре с типове, които моделират ownership

В модерния C++: raw pointers ≠ ownership

Пример за клас School

```
class School {
    std::vector<Student*> students_;

public:
    ~School() {
        for (Student* s : students_)
            delete s;
    }

    // Добавяне чрез копие
    void addStudent(const Student& s) {
        students_.push_back(new Student(s));
    }

    // Добавяне чрез move
    void addStudent(Student&& s) {
        students_.push_back(new Student(std::move(s)));
    }

    void print() const {
        for (const Student* s : students_)
            s->print();
    }
};
```

Пример за клас School на Java

```java
import java.util.ArrayList;
import java.util.List;

public class School {
    private List<Student> students = new ArrayList<>();

    // Добавяне чрез референция (НЕ копира обекта!)
    public void addStudent(Student s) {
        students.add(s);
    }

    // Явно копиране (по желание)
    public void addStudentCopy(Student s) {
        students.add(new Student(s));
    }

    public void print() {
        for (Student s : students)
            s.print();
    }
}
```

Въпроси?

Благодаря за вниманието!